Machine Learning Homework 1 Report

Student Information

Name: Adilhan Çetin
Student Number: 2580421

Question 1

a)

The model achieved the following performance metrics on the test set:

- **Accuracy:** 0.9362
- **Precision (Macro):** 0.8719
- **Recall (Macro):** 0.8860
- **F1-Score (Macro):** 0.8787

The model demonstrates strong overall predictive capabilities, achieving a high accuracy of roughly 93.6%. While the accuracy is excellent, the precision, recall, and F1-scores (around 87-88%) offer a more nuanced view of the model's performance. Because the *Macro* averaging method was used, we can confidently say the model generalized well across all class labels—meaning it successfully identified the minority classes rather than strictly predicting the majority class. The minor gap between accuracy and the macro metrics suggests a slight imbalance in the dataset, but an F1 score of 0.8787 indicates a very healthy balance between false positives and false negatives.

b)

For this problem, I implemented the ID3 Decision Tree algorithm entirely from scratch, adapting the pseudocode provided in the lecture slides to handle categorical features. My setup relies on a recursive `id3_training_loop` that first checks for base stopping cases (uniform labels, empty dataset, or no remaining attributes) to assign majority-vote leaf nodes. If the base cases are not met, it calculates the Information Gain for all available features

using a custom entropy function, selects the feature that maximizes this gain, and dynamically generates multi-way branches for every possible value of that feature by calling itself recursively. After the tree is built, predictions are made by tracing downward from the root, and I calculate the macro accuracy, precision, recall, and F1 metrics manually to evaluate the multi-class performance.

c)

Since the full tree generates 238 distinct rules, I added an `extract_rules` function in the code to trace and retrieve every path programmatically. Below is a concise excerpt of rules generated directly from our learned ID3 tree, showing examples for `unacc`, `acc`, and `vgood` classes:

$$\begin{aligned}\text{Prediction(unacc)} \iff\ & (\text{safety} = \text{low}) \vee \\ & (\text{safety} = \text{med} \wedge \text{persons} = 2) \vee \\ & (\text{safety} = \text{high} \wedge \text{persons} = 2)\end{aligned}$$

$$\begin{aligned}\text{Prediction(acc)} \iff\ & (\text{safety} = \text{high} \wedge \text{persons} = 4 \wedge \text{buying} = \text{low} \wedge \text{maint} = \text{vhigh}) \vee \\ & (\text{safety} = \text{med} \wedge \text{persons} = 4 \wedge \text{buying} = \text{low} \wedge \text{maint} = \text{vhigh} \wedge \text{lug\_boot} = \text{big})\end{aligned}$$

$$\text{Prediction(vgood)} \iff (\text{safety} = \text{high} \wedge \text{persons} = 4 \wedge \text{buying} = \text{low} \wedge \text{maint} = \text{low} \wedge \text{lug\_boot} = \text{big})$$

Question 2

a)

I trained two separate decision tree models using different splitting criteria. Here is how they performed on the test set:

- **Gini Criterion:** Accuracy: 0.8917 — Precision: 0.9089 — Recall: 0.9088 — F1-Score: 0.9088
- **Entropy Criterion:** Accuracy: 0.8975 — Precision: 0.9102 — Recall: 0.9103 — F1-Score: 0.9102

Both models handled the test data very well, scoring around 89–91% across the board. The Entropy model slightly outperformed the Gini model, pulling ahead by about 0.6% in accuracy. This makes sense, as Entropy tends to naturally produce slightly more balanced trees. More importantly, the consistently high macro F1-scores tell us that both models successfully learned to identify the minority bean classes rather than just taking the easy route of predicting the most common ones.

b)

For this task, I used the `scikit-learn` library to build the decision trees and the `ucimlrepo` package to cleanly pull in the Dry Bean Dataset.

First, I split the data into 80% training and 20% test sets. To make sure no bean class was accidentally underrepresented in either set, I used a stratified split (`stratify=y`). I also set a fixed `random_state=42` everywhere as stated in the homework instructions. I have not used in hyperparameter tuning. Finally, I evaluated them using `scikit-learn`'s built-in metrics.

Question 3

a)

The `sklearn.tree.DecisionTreeClassifier` uses an optimized version of the CART (Classification and Regression Trees) algorithm. The largest difference from our ID3 implementation is that CART handles continuous, numeric features natively by dividing data at optimal threshold values ($x \leq$ threshold) to create strictly binary splits, rather than making multi-way branches for every category. Furthermore, while ID3 relies purely on entropy for classification tasks, CART supports both classification and regression, and gives you the choice to split based on either Gini Impurity (its default) or Entropy.

c)

The best approach would be to use a algorithm similar to binary search. Even tough we have a continous numeric feature, we can create finite number of splits by finding the optimal threshold values. We can sort the data based on the feature we want to split on and then evaluate potential splits at each unique value of that feature. This way, we can effectively handle continuous features while still maintaining the decision tree structure.